```
import numpy as np
a= np.array([[4,5,3],[2,1,3],[4,2,1]])
print("maximum:", a.max())
print("minimum:", a.min())
print("mean", a.mean())
print("median", np.median(a))
print("standard", a.std())
print("variance", a.var())
print("determinant", np.linalg.det(a))
print("rank", np.linalg.matrix.rank(a))
print(np.linalg.inv(a))
print(np.linalg.eig(a))
```

O/P →

```
maximum : 5
minimum : 1
mean : 2.7777
median : 3.0
Standard : 1.31468
variance : 1.72839506
determinant : 30.0000
rank : 3
inverse : [[-0.166667   0.33333   0.4]
           [0.33333   -0.2666667   -0.2]
           [0.          0.4         -0.2]]
eigen : ([-1.30552558+1.3339783 2j , -1.30552558-
          1.3397832j])
```

(2) import pandas as pd

import numpy as np

```
data = pd.DataFrame({"id":[102,101,103,np.nan],
"name":["abc","bch",np.nan,"rst"],
"age":[np.nan,np.nan,23,19],"marks":[98,np.
nan,45,87]})

print(data)
print(data.tail(2))
print(data.info())
print(data["name"].isna())
print(data["name"].isna().sum())
print(data.fillna(0))
print(data["marks"].mean())
```

O/P →

	id	name	age	marks
0	101.0	abc	NaN	98.0
1	102.0	bch	NaN	NaN
2	103.0	NaN	23.0	45.0
3	104.0	rst	19.0	87.0

	id	name	age	marks
2	103.0	NaN	23.0	45.0
3	NaN	rst	19.0	87.0

	id	name	age	marks
0	101.0	abc	NaN	98.0
1	102.0	bch	NaN	NaN
2	103.0	NaN	23.0	95.0

0 False

1 False

2 True

3 False

Name: name, dtype: bool

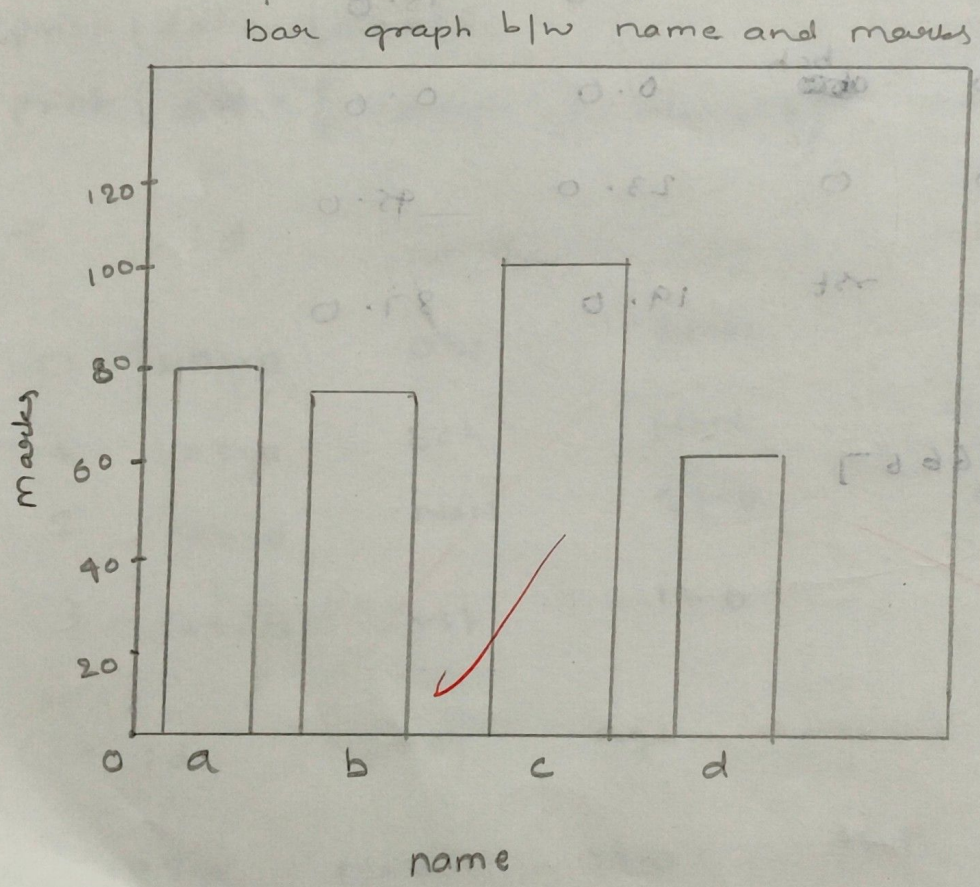
	id	name	age	marks
0	101.0	abc	0.0	98.0
1	102.0	bch	0.0	0.0
2	103.0	0	23.0	95.0
3	0.0	rst	19.0	87.0

76.66666667

(3)

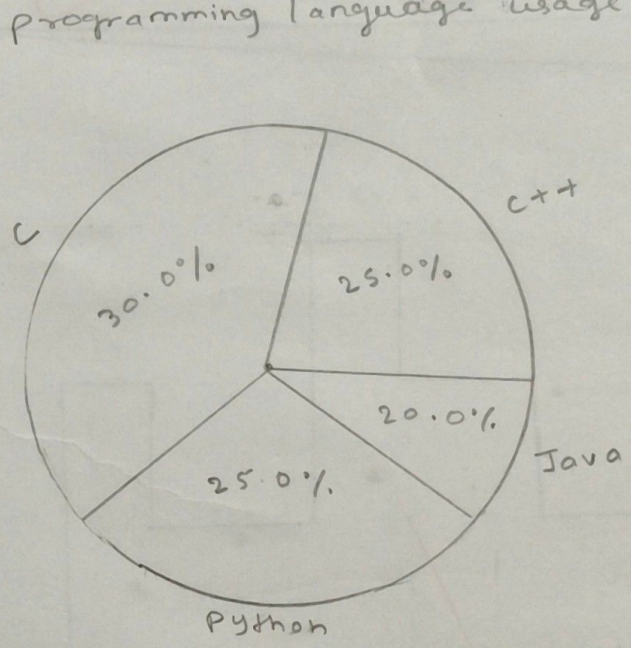
Bar graph

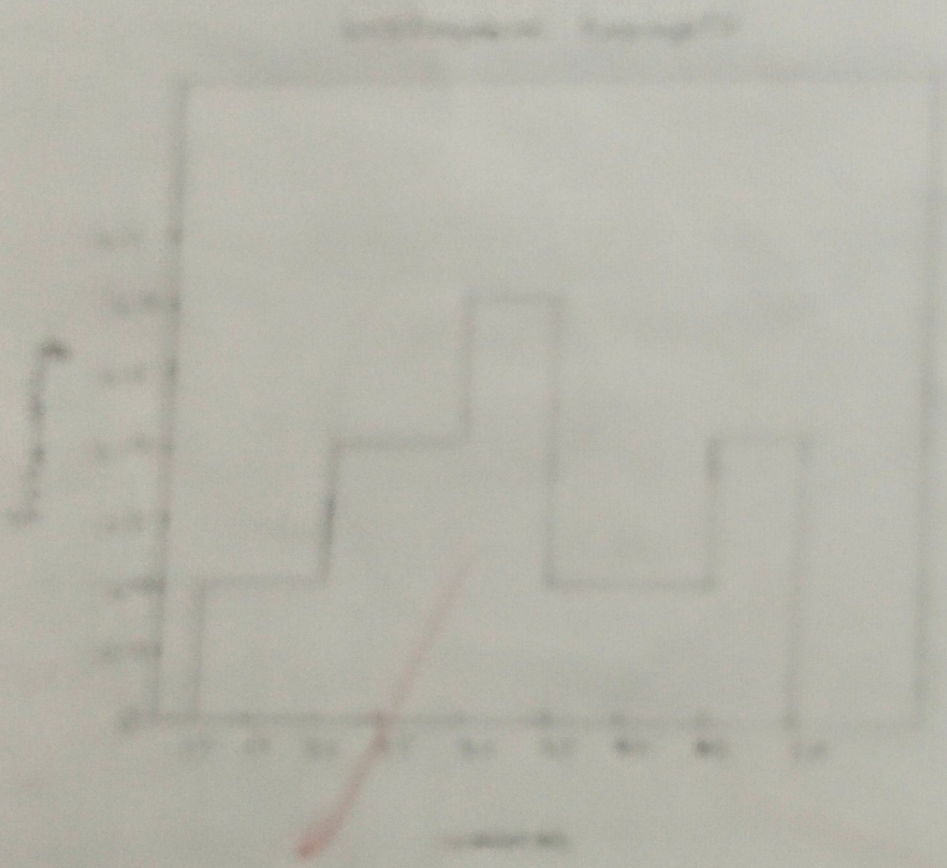
```
import matplot.pyplot as plt
name = ["a", "b", "c", "d"]
marks = [87, 78, 98, 56]
plt.bar(name, marks)
plt.xlabel("name")
plt.ylabel("marks")
plt.title("bar graph b/w name and marks")
plt.show()
```



Pie chart

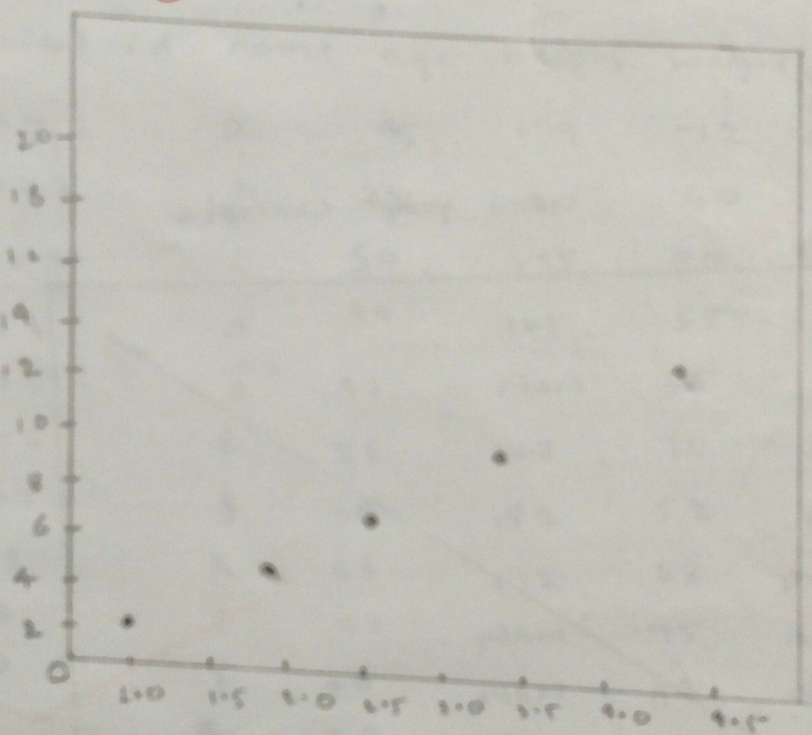
```
import matplotlib.pyplot as plt
lang = ["c", "python", "Java", "c++"]
size = [30, 25, 20, 25]
plt.pie(size, labels = lang, autopct = "%.1f %%",
        startangle = 90)
plt.title("programming language usage")
plt.show()
```





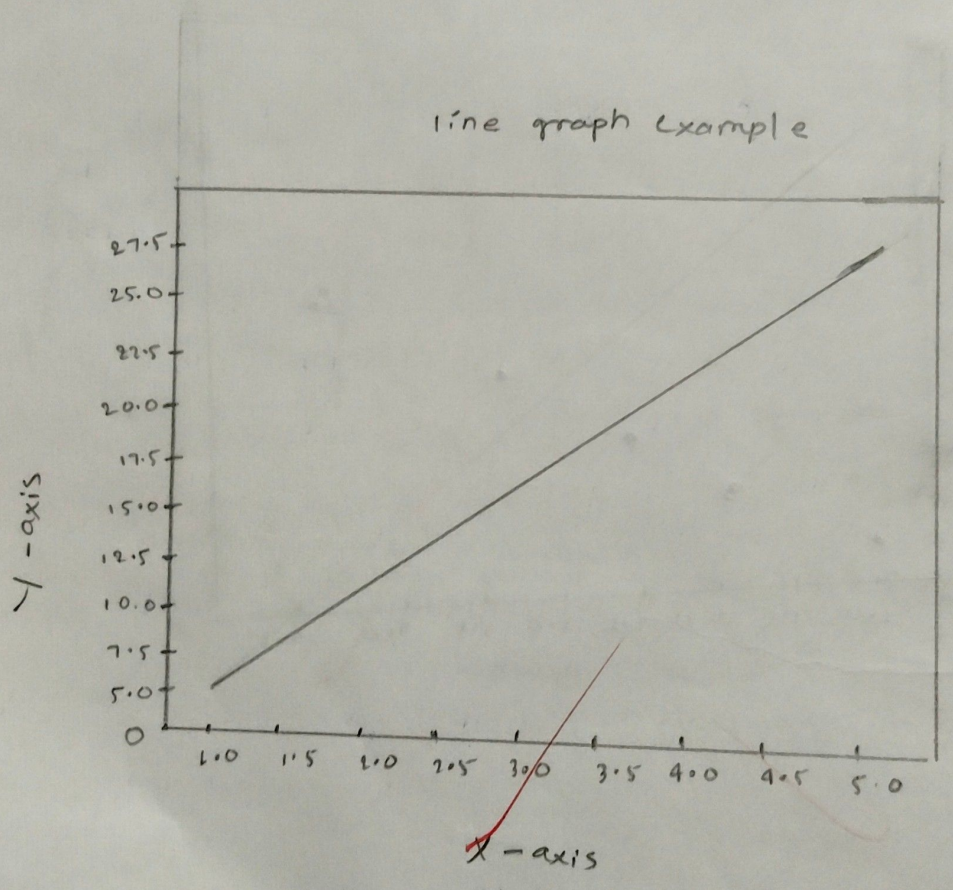
Scatter plot

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.title("Scatter plot example")
plt.scatter(x, y)
plt.xlabel("x values")
plt.ylabel("Y values")
plt.show()
```



Line graph

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [5, 10, 15, 20, 25]
plt.plot(x, y)
plt.title("line graph example")
plt.xlabel("x-axis")
plt.ylabel("Y-axis")
plt.show()
```



Q) Create a dataset of patient health care information which contain columns such as patient id, name, height, weight, diagonosis, Sugar level, by the help of numpy, pandas, matplotlib, display detail like std mean, description deviation, size dimensions, head, tail.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
data = pd.read_csv("patient.csv")
print(data)
```

O/P

	S.No.	Patient-id	name	age	height	weight	diagonis	Sugar level
0		P001	a	45	179	72	diabetes	180
1		P002	b	38	160	60	NaN	110
2		P003	c	50	175	80	normal	200
3		P004	d	39	165	55	diabetes	NaN
4		P005	e	41	NaN	65	NaN	NaN
5		P006	f	36	168	70	normal	115
6		P007	g	28	182	52	NaN	NaN
7		P008	h	54	172	68	prediabetes	140
8		P008	i	33	NaN	75	normal	190
9		P010	j	40	NaN	64	diabetes	NaN
10		P011	K	42	156	51	[illegible] NaN	NaN
11		P012	l	50	NaN	62	NaN	NaN
12		P013	m	39	165	49	[illegible] diabetes	210
13		P014	n	37	NaN	48	normal [illegible]	105
14		P015	o	28	NaN	58		NaN
15		P016	p	41	160	65	NaN	200
16		P017	q	51	155	67	prediabetes	180
17		P018	r	55	175	77	NaN	110
18		P019	s	67	NaN	47	NaN	115
19		P020	t	32	NaN	56	NaN diabetes	NaN

print("head:", data.head())

o/p	patient id	name	age	height	weight	diagonis	sugar level
0	P001	a	45	179	72	diabetes	180
1	P002	b	38	160	60	NaN	110
2	P003	c	50	175	80	normal	200
3	P004	d	29	155	55	diabetes	NaN
4	P005	e	41	NaN	65	NaN	NaN

print("tail", data.tail())

o/p	patient-id	name	age	height	weight	diagonis	sugar level
15	P016	p	45	160	77	prediabetes	200
16	P017	q	51	155	74	NaN	180
17	P018	r	65	175	62	NaN	110
18	P019	s	67	NaN	75	NaN	115
19	P020	t	32	NaN	80	diabetes	NaN

print("size", data.size())

o/p - size : 140

print("shape", data.shape)

o/p shape : (20, 7)

print("mean", data.mean(numeric_only=True))

o/p - mean: age 43.050000
height 165.333333
weight 67.350000
sugar level 154.558330
dtype: float64

```
print("standard deviation:", data.std(numeric_only=True))
```

O/P –

```
Standard deviation:  age           9.843
                     height        8.315
                     weight        9.598
                     sugar level   42.128
                     dtype: float64
```

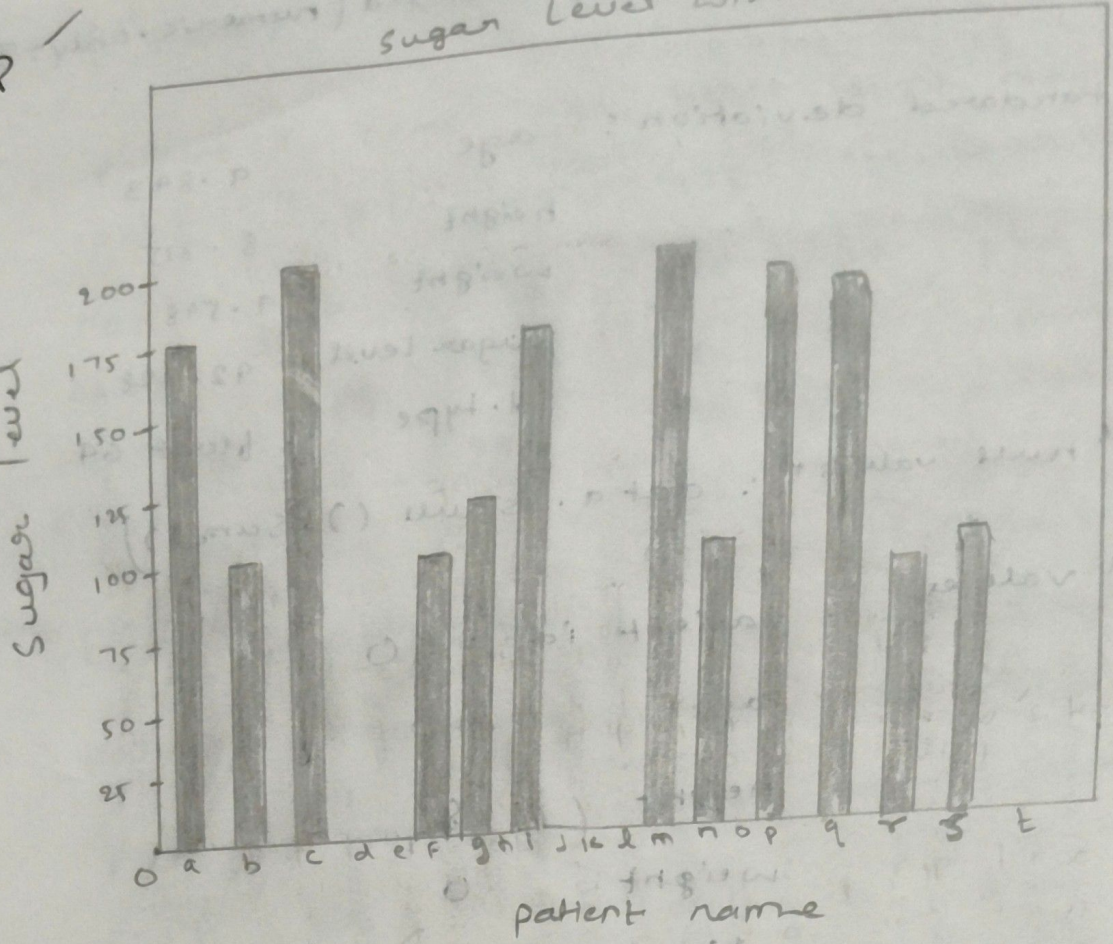
```
print("null values:", data.isnull().sum())
```

O/P – null values : patient id : 0

```
age           0
height        8
weight        0
diagonasis    9
sugar level   8
dtype: int64
```

```
plt.bar(data["name"], data["sugar level"])
plt.xlabel("patient name")
plt.ylabel("sugar level")
plt.title("sugar level with null values")
plt.show()
```

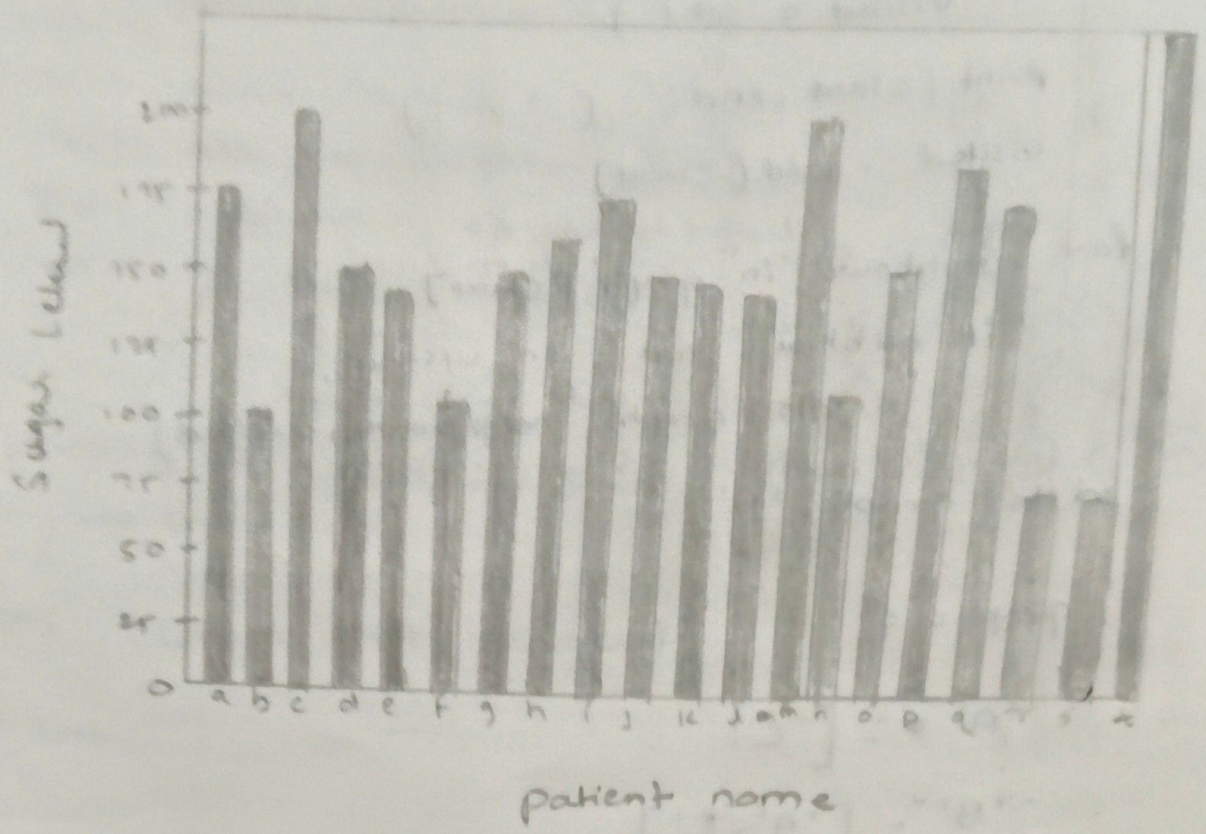
O/P




```
data["sugarlevel"].fillna(data["sugarlevel"].mean
(), inplace = True)
data["height"].fillna(data["height"].mean(),
inplace = True)
plt.bar(data["name"], data["sugarlevel"])
plt.xlabel("patient name")
plt.ylabel("sugar level")
plt.title("sugar level with replace null values")
plt.show()
```

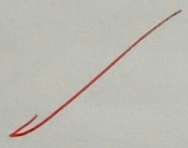
o/p-

sugar level replace with null values



```
def dfs(graph, start, visited=None):
    if visited == None:
        visited = set()
    print(start, end='')
    visited.add(start)

for neighbour in graph[start]:
    if neibhour not in visited:
        dfs(graphs, neighbour, visited)

dfs(graph, 'A')

graph = {
    'A': ['C'
    'B': ['A', 'E'],
    'C': ['F'],
    'D': ['G'],
    'E': [],
    'F': [],
    'G': [],
```

```
from collections import deque
def Bfs( graph, start):
    visited = set( )
    queue = deque ([start])
    while queue:
        node= queue. popleft( )
        if node not in visited:
            print (node, end= " ")
            visited. add (node)

        for neighbor in graph [node]:
            if neighbor not in visited:
                queue. append (neighbor)
```



Breadth – first search (BFS)

idea : → Explore level by level → closest thing first

Real - life applications

1) shortest path in maps and GPS
- finding the minimum number of roads/steps between two places
- used when all roads have equal cost

2) Social network
- People you may know
- finding friends within 1 hop, 2 hops etc

3) Web Cr
- Search engines indexing websites level by level fr a starting page

4) Network broadcasting
- Sending data to all computers in a network with minimum delay.

5) Puzzle and game solving
- finding the minimum moves to solve puzzles (eg. chess Rubiks Cube states)

Depth – first search (DFS)

Idea : Go deep first, then backtrack

Real - life applications:

1. File System traversal
- Exploring folders inside folder on your computers

2. Maze solving
- Go as far possible down one path, then backtracks

3) Detecting cycles
- used in deadlock detection and dependency checking

4) Topological sorting
- Task scheduling (eg. prerequisites in coorses)

5) AI and game decision trees
- Exploring possible moves deeply in games like chess

Q) What is Bfs and Dfs:

BFS = Breadth first search

It is a algorithm used to traverse or search a graph or tree
- Start from a node
- visit all it nearest neighbour first
- Then move to the next level

→ It explores level by level

Data structure used -

2) Dfs - Depth first search

It is also an algorithm to traverse or search a graph or tree.
- Start from a node

Go as deep as possible along one path
when stuck, backtrack and try another path

→ It explores deep first.

Data structure used → Stack (or recursion)

) Why are we studying Bfs and Dfs

→ BFS :-

Shortest path, find minimum hops between two nodes
connected components . bipartite check, web crawling
network broad casting

Dfs:-

→ Pathfinding and backtracking - Maze puzzle

→ cycle detection, topological sorting

→ Memory efficiency

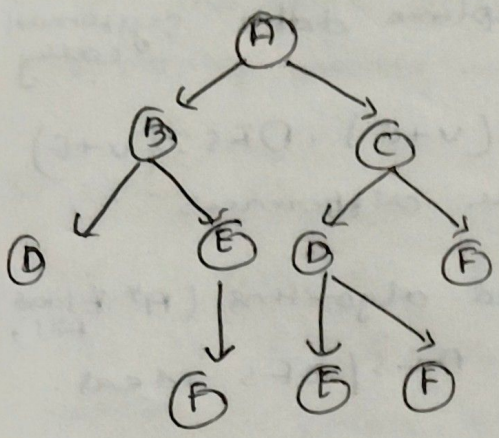
1) Algorithmic thinking:- how to explore data systematically

2) Time and space complexity - BFS: O(V+E), DFS: O(V+E) but stack + queue differences

3) Problem patterns: Many advanced algorithms (A* Flood Fill, union - find alternatives) build on BFS/DFS ideas.

(1) Design and Develop Depth first search (DFS) for the following

1) Construct a static graph consisting of 6 nodes and 10 edges where the graph is stored in dictionary representation



A-B
A-C
B-D
B-E
C-D
C-F
D-E
D-F
E-F
B-F

2) Create the graph dynamically where the values of graph is given by the user during runtime

3) Accept the root node and end node from user

4) finally display the root node and end node

5) Same graph from bfs.

1>
```
def dfs( graph , start , visited = None):
    if visited is None:
        visited = set ( )
    print ( start , end = ' ' )
    visited. add (start)
    for neighbour in graph [start]:
        if neighbour not in visited:
            dfs (graph , neighbour, visited)
dfs (graph , 'A')
graph = { 'A' : ['B', 'C'],
          'B' : [ 'D' , 'E', 'F'],
          'C' : ['D' , 'F' ]
          'D' : ['E' , 'F' ]
          'E' : [ ],
          'F' : [ ] }
```

(2)
```
def dfs (graph , start , visited = none):
    if visited is None:
        visited = set ( )
    print (start, end = " ")
    visited . add (start)
    for neighbour is graph [start]:
        if neighbour not in visited:
            dfs ( graph, neighbour, Visited)
graph= { }
nodes = int (input ("enter node"))
edges = int ( input ("enter edge"))
print ("enter node name")
```

```
for i in range (nodes):
        node = input ( )
        graph = [node] = [ ]
Print ("enter edges")
for i in range (edges):
        u, v = input . split ( )
        graph [u]. append (v)
        graph [v]. append (u)
st = input ("enter starting node")
dbs (graph, st)
```

I/P

- enter nodes : 4
  enter edges : 3

enter node name

A

B

C

D

enter edges

A B

A C

B D

enter starting node:

A

O/P ⇒ A B D C

③

```
def dfs(graph, start, end, visited=None):
    if visited is None:
        visited = set()
    print(start, end="  ")
    visited.add(start)
    if start == end:
        return True
    for neighbour in graph[start]:
        if neighbour not in visited:
            if dfs(graph, neighbour, end, visited):
                return True
    return false

graph = { }
nodes = int(input("enter nodes"))
edges = int(input("enter edges"))
print("enter node name")
for i in range(nodes):
    node = input()
    graph[node] = []
print("enter edges")
for i in range(edges):
    u, v = input.split()
    graph[u].append(v)
st = input("enter initial node")
end = input("enter final node")
found = dfs(graph, st, end)
```

I/p ⇒ enter nodes : 5
enter edges : 4
enter node name

A
B
C
D
E

enter edges

A B

A C

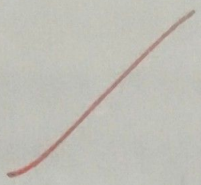
B D

C E

enter initial node : A

enter final node : E

o/p → A B D C E



~~Instruction for Team work~~

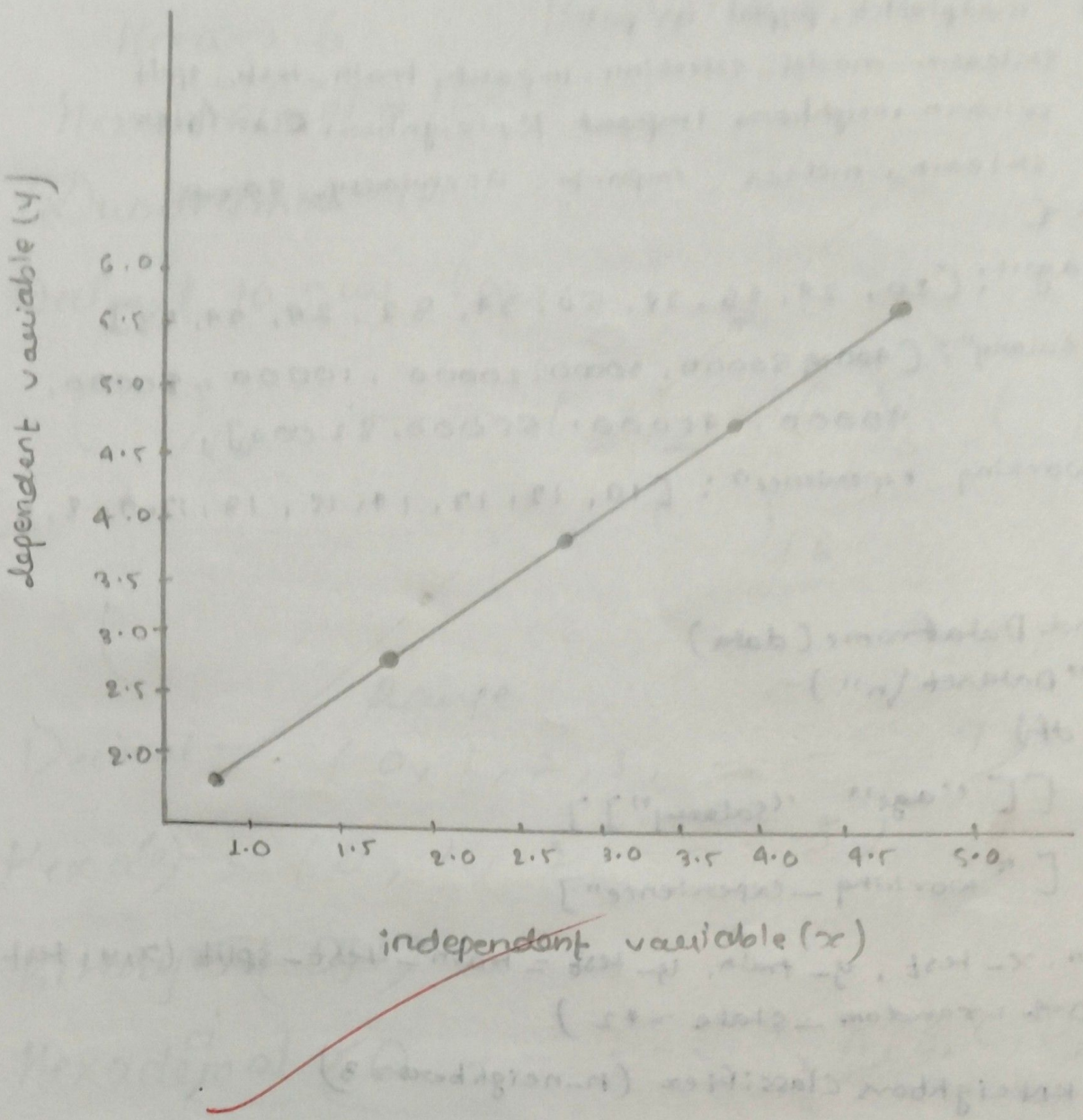
Normalization and Standardization

```
import pandas as pd
data = pd.DataFrame({"name": ["A", "B", "C", "D",
                              "E"],
                     "age": [23, 21, 18, 25, 29],
                     "Height": [154, 163, 145, 134, 175],
                     "Income": [36000, 40000, 10000, 15000]})
print(data)

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
data[['age', 'income']] = scaler.fit_transform(data[[
                                'age', 'income']])
print(data)

from sklearn.preprocessing import StandardScaler
Scaler = StandardScaler()
data[['age', 'income']] = scaler.fit_transform(data
                                [['age', 'income']])
print(data)
```

Linear Regression

```
from sklearn.metrices import mean_absolute_error,
                             mean_squared_error
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
x= np.array([[1],[2],[3],[4],[5]])
y= np.array([[2],[3],[4],[5],[6]])
model= LinearRegression()
X_train, y_train, x_test, y_test = train_test_split(x,y,test_
                                   size=0.2, random_state=42)
model.fit(x,y)
y_pred= model.predict(x_test)
print(model.coef_)
print(model.intercept_)
plt.scatter(x,y, color="blue")
plt.plot(x,y, color="red")
plt.xlabel("independent variable(x)")
plt.ylabel("dependent variable(y)")
plt.show()
```

→ op→ [[1.]]
[1.]


dependent variable (y)
6.0
5.5
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
independent variable (x)

30/03/26